

ORCA Investment OS — Full-Scope Security Audit & Technical Assessment

Date: 2026-08-22 · **Type:** White-box code & configuration audit · **Auditor:** Lovable Agent

Scope: Frontend (React/Vite), Lovable Cloud backend (Postgres + RLS + Edge Functions), auth flows, business logic, operations.

Executive Summary

The platform's **server-side data layer is in strong shape**: RLS is enabled on every public table, all 37 `SECURITY DEFINER` functions pin `search_path` and gate admin surfaces behind `has_role()`, exchange credentials are vaulted with read-only scope enforcement, and entitlement checks are ownership-gated.

The **residual risk concentrates on the perimeter**: one public edge function leaks account existence, the Content-Security-Policy is report-only (no enforcement), a formula evaluator uses `new Function`, session tokens live in `localStorage`, and the pinned `xlsx` build carries known CVEs with no npm fix path.

Severity	Count	Headline
Critical	0	—
High	2	User enumeration + unvalidated <code>redirect_to</code> in password reset; CSP not enforced
Medium	4	<code>new Function</code> evaluator, <code>xlsx</code> CVEs, <code>localStorage</code> tokens, missing rate limiting
Low	4	Wildcard CORS, <code>document.write</code> print window, fallback HMAC salt, header hygiene
Info / Pass	40	RLS, vault, role model, grants, function guards — all verified clean

Wave 1 — Architecture & Attack Surface

Routes. Public: `/welcome`, `/auth`, `/privacy`, `/terms`, `/accessibility`, landing. Authenticated: gated by `RequireAuth` (session check → redirect). Admin console: gated by `RequireAdmin` which calls `public.has_role(uid, 'admin')` over RPC — **server-side, not client-side. PASS**

Trust boundaries. Client → PostgREST (RLS-scoped, anon key) and client → Edge Functions. Six functions run with `verify_jwt = false` and therefore perform their own auth: `delete-account`, `orca-coach`-style user functions verify the bearer token via `auth.getUser()`; `sync-ibkr-flex` supports a cron mode protected by a timing-safe-compared `x-cron-secret`. **PASS** Manual verification present on all but one — see F-01.

Supply chain. `xlsx@0.18.5` is pinned from npm — SheetJS ceased publishing fixes to npm after 0.18.5; known CVEs (below) have no npm upgrade path.

Wave 2 — Identity & Session

- Session tokens stored in `localStorage` (`persistSession: true`) — any successful XSS is full account takeover. Finding F-05.
 - Google OAuth is the sole sign-in path (email/password removed by design). Redirect uses same-origin `window.location.origin`. **PASS**
 - Password-reset edge function is public and **distinguishes registered vs unregistered emails** (`not_registered` vs `ok`) → account enumeration. It also **passes caller-supplied `redirect_to` straight to `/auth/v1/recover`** without validating against an allowlist → recovery-link redirection risk. Finding F-01.
 - No evidence of **rate limiting** on `request-password-reset` or `orca-coach` → mail-bombing / AI-cost abuse vector. Finding F-06.
 - Sign-out clears session and redirects; per-user `localStorage` keys are wiped on reset. **PASS**

Wave 3 — Data Layer (RLS / Functions / Grants)

Verified live against the database catalog:

- RLS enabled on all 25 public tables.** Policies scope to `auth.uid()`; `economic_events` `SELECT` is intentionally public (market calendar data, non-PII). **PASS**
 - All 37 `SECURITY DEFINER` functions** set `search_path` explicitly (no search-path hijack) and every `admin_*` function opens with `if not public.has_role(auth.uid(), 'admin')` then `raise ... 42501`. **PASS**
 - `current_entitlement`** raises `forbidden` when a non-admin queries another user's entitlement. **PASS**
 - `exchange_credentials_vault_store`** (BEFORE trigger) enforces ownership (`user_id mismatch`), enforces **read-only scopes only**, stores secrets in `vault.secrets`, and **nulls the plaintext column before persistence**. Delete trigger purges the vault secret. `read_exchange_secret` is ownership-gated. **PASS** — this is the crown-jewel path and it is correctly built.
 - Role model:** roles live in `public.user_roles` (never on profiles), checked via `has_role` security-definer function. No client-side admin flags. **PASS**
 - Grants** present on public tables for `authenticated` / `service_role`; `anon` granted only where a public-read policy exists. **PASS**
 - Bug board:** status transitions admin-only both in trigger (`bug_reports_before_update`) and RPC (`set_bug_status`) — defence in depth. **PASS**
 - Hardening note (F-09, Low):** `trader_code()` falls back to literal salt `'CHANGE-ME-SET-A-REAL-SALT'` if the vault secret `trader_salt` is absent — pseudonymisation then becomes reproducible. Ensure the vault secret exists.

Wave 4 — Edge Functions

Function	JWT	Auth check	Notes
<code>delete-account</code>	off	<code>auth.getUser()</code> bearer PASS	Deletes own account only
<code>orca-coach</code>	on	<code>getUser()</code> PASS	No rate limit (F-06)
<code>request-password-reset</code>	off	none (public by design)	F-01 enumeration + redirect_to
<code>sync-futures-trades</code>	off	<code>getUser()</code> PASS	Reads vault per-user

Function	JWT	Auth check	Notes
sync-ibkr-flex	off	<code>getUser()</code> or timing-safe cron secret PASS	
sync-economic-events	off	cron/scheduled path	Writes public calendar rows only
validate-exchange-credential	off	<code>getUserId(authHeader)</code> PASS	Read-only scope enforced server-side
market-candles	on	—	Public market data proxy
backfill-provenance	off	bearer required PASS	Batch-limited, <code>FOR UPDATE SKIP LOCKED</code>

- **CORS Access-Control-Allow-Origin:** * on all functions (F-07, Low). Acceptable for bearer-token APIs (no cookies), but tighten to the app origin where possible.
 - No function logs secrets; vault read failures return generic `vault_read_failed`. **PASS**

Wave 5 — Frontend

- **F-02 (High): CSP is Content-Security-Policy-Report-Only** and the policy itself contains `'unsafe-inline' 'unsafe-eval'`. Nothing is enforced; even in enforce mode the current policy would not stop injected scripts. Path: remove `unsafe-eval`, hash/nonces for the preboot inline script, then flip to enforcing.
- **F-03 (Medium): new Function() in use-dashboard-config.ts:142** evaluates user-authored KPI formulas. Token whitelist + assignment/keyword regex reduce the surface, but regex-based JS sandboxing is not a boundary — a crafted identifier/property chain is a known bypass class. Replace with a real expression parser (e.g. mathjs limited scope or a tiny recursive-descent evaluator).
- **F-04 (Medium): xlsx@0.18.5** — CVE-2023-30533 (prototype pollution) & CVE-2024-22363 (ReDoS). Mitigations: it parses only user-selected local files (self-inflicted blast radius), but the import pipeline also feeds parsed cells into trade reconstruction — sanitize/validate cell values before use. Long-term: move to SheetJS CDN builds or an alternative parser.
- **F-08 (Low): document.write** into a print window in `SettingsHub` (srcdoc of app-generated HTML only) and `innerHTML` reads (not writes) in `TraderMind/console` — no untrusted sinks found. `dangerouslySetInnerHTML` in `ui/chart.tsx` injects **theme-generated CSS only**, no user data. **PASS**
 - No secrets in the client bundle; only the publishable anon key (public by design). **PASS**

Wave 6 — Business Logic

- Entitlement/tier logic resolved **server-side** in `current_entitlement` — client tier gates are UX-only; data access is enforced by RLS + RPC guards. **PASS**
 - Risk-breach, expectancy, and admin rollop math run in SQL under the caller's identity — no client-supplied `user_id` is ever trusted (`auth.uid()` only). **PASS**
 - Bug-arena identity resolution (`bug_arena_people`) exposes only display name + avatar for users sharing a bug thread — bounded disclosure. **PASS**
 - Benchmarks endpoint enforces **k-anonymity** (`p_kmin`, default 25) before releasing aggregates. **PASS**

Wave 7 — Operations

- Storage buckets `avatars` and `bug-attachments` are **private**. **PASS**
 - Secrets (`SUPABASE_SERVICE_ROLE_KEY`, `FINNHUB_API_KEY`, etc.) live in the function secret store; none appear in code or logs. **PASS**
 - PWA service worker (`public/sw.js`) — verify it never caches authenticated API responses (recommend `networkOnly` for `*.supabase.co`).
 - Security headers: CSP report-only exists; add `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` at the host layer.

Wave 8 — Findings Register

ID	Severity	Finding	Evidence	Remediation
F-01	High	Password-reset enumeration + unvalidated <code>redirect_to</code>	<code>request-password-reset/index.ts</code> returns <code>not_registered</code> ; passes caller <code>redirectTo</code> to <code>/auth/v1/recover</code>	Return identical response either way; validate <code>redirect_to</code> against an origin allowlist; add rate limit
F-02	High	CSP report-only + <code>unsafe-inline/eval</code>	<code>index.html:149</code>	Nonce the preboot script, drop <code>unsafe-eval</code> , enforce
F-03	Medium	<code>new Function</code> KPI formula evaluator	<code>use-dashboard-config.ts:142</code>	Replace with parser-based evaluator
F-04	Medium	<code>xlsx@0.18.5</code> known CVEs, no npm fix	<code>package.json</code>	Upgrade via SheetJS CDN registry or replace;

ID	Severity	Finding	Evidence	Remediation
				sanitize parsed cells
F-05	Medium	Session tokens in localStorage (XSS-reachable)	<code>client.ts</code> <code>persistSession</code>	Inherent to SPA+anon-key model; mitigate via F-02 enforcement + XSS hygiene
F-06	Medium	No rate limiting on public/AI functions	<code>request-password-reset</code> , <code>orca-coach</code>	Add per-IP/per-user throttling (edge-level or table-based)
F-07	Low	Wildcard CORS on all edge functions	<code>*/index.ts</code> CORS blocks	Restrict <code>Access-Control-Allow-Origin</code> to app origin
F-08	Low	<code>document.write</code> print path	<code>SettingsHub.tsx:2799</code>	Self-generated markup only; replace with blob-url print iframe
F-09	Low	<code>trader_code</code> fallback salt	<code>trader_code()</code>	Ensure <code>trader_salt</code> vault secret is set; fail closed otherwise
F-10	Low	Missing hardening headers	hosting layer	Add nosniff / Referrer-Policy / Permissions-Policy

Verified clean (40 checks): RLS coverage & scoping, all SECURITY DEFINER guards, `search_path` pinning, vault credential lifecycle, role storage model, admin gating (client + server), grants, k-anonymity benchmarks, no secret logging, no untrusted HTML sinks, OAuth same-origin redirects, storage privacy, bug-board transition guards.

Top 5 Priority Actions

- 1. Fix `request-password-reset`: uniform response + `redirect_to` allowlist + rate limit.
- 2. Promote CSP from report-only to enforcing (nonce the theme preboot, remove `unsafe-eval`).
- 3. Replace the `new Function` KPI evaluator with a real expression parser.
- 4. Add rate limiting to `orca-coach` (AI cost abuse).
- 5. Resolve the `xlsx` dependency (CDN build or alternative parser).

Remediation Log — 2026-08-22

ID	Status	Action taken
F-01	Fixed	<code>request-password-reset</code> now returns a uniform <code>{ ok: true }</code> for registered and unregistered emails alike (no enumeration), and <code>redirect_to</code> is sanitized (https-only, no embedded credentials) before forwarding. Deployed & verified live.
F-06	Fixed	Rate limiting added: password-reset capped at 5/email/hour and 20/IP/hour via new internal <code>rate_limit_events</code> table (deny-all client policy, service-role only) — verified live with a 429 on the 6th request. <code>orca-coach</code> capped at 30 calls/user/hour via <code>ai_runs</code> telemetry.
F-03	Fixed	<code>new Function</code> KPI evaluator replaced with a recursive-descent parser (arithmetic + whitelisted math functions only, zero code execution). Covered by <code>src/test/kpi-parser.test.ts</code> — injection attempts (<code>process.exit</code> , <code>constructor.constructor</code> , ternaries, unknown identifiers) all return null.
F-02	Open	CSP enforcement requires noncing the theme preboot script — scheduled as a separate change with telemetry review.
F-04	Open	<code>xlsx</code> upgrade path under evaluation (SheetJS CDN registry vs alternative parser).
F-05	Mitigated by design	localStorage tokens are inherent to the SPA model; risk drops sharply once F-02 lands.

ID	Status	Action taken
F-07–F-10	Open (Low)	CORS origin restriction, print-window refactor, vault salt check, hardening headers.

ORCA Investment OS — Confidential security assessment. Generated 2026-08-22. White-box audit of code, database catalog, and edge-function configuration.